

Relatório Técnico – Equipe 5 (String Match)

Descrição do Algoritmo

1. Algoritmo Implementado

1.1 Visão Geral

O projeto implementa o algoritmo de busca quântica de padrões proposto por Niroula & Nam (2021), publicado no npj Quantum Information. Este algoritmo resolve o problema de exact string matching em um texto binário utilizando a busca de Grover, obtendo uma aceleração quadrática em relação aos melhores algoritmos clássicos.

O problema é: Dado um texto binário T , de comprimento N , e um padrão binário P , de comprimento M ($M \leq N$), encontrar todas as posições i onde $T[i .. (i + M - 1)] = P$.

1.2 Fundamentação Teórica

Diferentemente de propostas anteriores (como Ramesh & Vinay, 2000), que descrevem apenas a complexidade de queries sem fornecer construção de circuito, Niroula & Nam apresenta uma implementação em nível de portas quânticas. As vantagens do algoritmo são:

Característica	Descrição
Complexidade de tempo	$O(\sqrt{N})$, aceleração quadrática
Complexidade de espaço	$O(N + M)$ qubits
Não requer QRAM	Elimina a necessidade de memória quântica de acesso aleatório
Circuito explícito	Construído com portas elementares (Hadamard, CNOT, Toffoli, Fredkin)

Tabela 1

1.3 Estrutura do Circuito Quântico

O circuito utiliza três registradores quânticos:

Registrador	Símbolo	Número de qubits	Finalidade
Índice	q_{idx}	$\lceil \log_2 (N-M+1) \rceil$	Superposição de todas as posições candidatas i
Texto	q_{text}	N	Codificação do texto binário T
Padrão	q_{pat}	M	Codificação do

			padrão P (usado para XOR depois)
--	--	--	----------------------------------

Tabela 2: Registradores do circuito quântico

1.4 Funcionamento Passo a Passo

Passo 1: Preparação do Estado

1. O registrador de texto é codificado com os bits de T utilizando portas X (NOT) nos qubits onde $T[j] = 1$.
2. O registrador de padrão é codificado analogamente com os bits de P.
3. O registrador de índice é levado a uma superposição uniforme mediante aplicação de portas de Hadamard:

$$|\psi_0\rangle = \frac{1}{\sqrt{N-M+1}} \sum_{i=0}^{N-M+1} |i\rangle \otimes |T\rangle \otimes |P\rangle$$

Passo 2: Deslocamento Cíclico Controlado

Para cada bit b do registrador de índice (representando a posição candidata i), aplica-se um deslocamento circular à esquerda no registrador de texto:

- Se o b-ésimo qubit de q_idx estiver em $|1\rangle$, o texto é deslocado por 2^b posições.
- O deslocamento total é, portanto, igual ao valor binário representado pelo registrador de índice: i.

Após esta operação, os primeiros M qubits do registrador de texto correspondem exatamente à substring $T[i..i+M-1]$.

O deslocamento é implementado utilizando portas CSWAP (Fredkin) organizadas em uma decomposição de permutação, resultando em profundidade $O(\log_2 N)$.

Passo 3: Comparação por XOR

A comparação entre a substring e o padrão é realizada aplicando portas CNOT:

$$\text{CNOT}(q_text[j], q_pat[j]) \text{ para } j = 0, \dots, M-1$$

O efeito desta operação é transformar o registrador de padrão em: $q_pat[j] = T[i+j] \oplus P[j]$

Assim, o registrador de padrão estará no estado $|00\dots 0\rangle$ se e somente se a substring for idêntica ao padrão.

Passo 4: Oráculo de Fase

O oráculo de fase aplica uma fase -1 exclusivamente ao estado $|00\dots 0\rangle$ do registrador de padrão. Sua implementação segue a sequência:

$$X^{\otimes M} \rightarrow H \rightarrow \text{MCX} \rightarrow H \rightarrow X^{\otimes M}$$

- Primeiro $X^{\otimes M}$: Inversão de todos os bits (converte $|00\dots 0\rangle$ em $|11\dots 1\rangle$).
- $H \rightarrow \text{MCX} \rightarrow H$: Implementa uma porta Z multi-controlada (MCZ).
- Último $X^{\otimes M}$: Restaura o registrador ao estado original, preservando a fase aplicada.

Passo 5: Descomputação (Uncompute)

Para garantir que o estado do circuito possa ser reutilizado na próxima iteração de Grover, é necessário desfazer as operações que modificam os registradores auxiliares:

1. Desfazer o XOR: Aplica-se novamente as mesmas portas CNOT (que são auto-inversas)
2. Desfazer o deslocamento cíclico: Aplica-se o deslocamento inverso. Para um deslocamento à esquerda por c , o inverso é o deslocamento à esquerda por $(N-c) \bmod N$

Passo 6: Correção de Estados Espúrios (Wrap-around)

Quando $N - M + 1$ não é uma potência de 2, o registrador de índice possui $2^{\lceil \log_2(N - M + 1) \rceil}$ estados, dos quais alguns representam posições inválidas. O deslocamento circular por essas posições pode criar matches "falsos" por wrap-around.

Para cada posição inválida que gera um match errado, aplica-se uma filtração para prevenir que o algoritmo faça um deslocamento para a posição errada.

Passo 7: Difusor de Grover

O difusor de Grover é o operador responsável por ampliar as amplitudes dos estados marcados (que receberam fase -1). Sua implementação no registrador de índice é:

$$H^{\otimes k} \rightarrow X^{\otimes k} \rightarrow \text{MCZ} \rightarrow X^{\otimes k} \rightarrow H^{\otimes k}.$$

Onde $k = \lceil \log_2(N - M + 1) \rceil$.

Passo 8: Iteração e Medição

Os passos 2 a 7 são repetidos por um número determinado de iterações:

- Para o caso automático: $R = \lfloor \frac{\pi}{4} \cdot \sqrt{\frac{2^{n_{idx}}}{n_{valid_matches}}} \rfloor$
- O número também pode ser fixado manualmente para estudo do fenômeno de super-rotação.

2. Variações de Entrada Implementadas

Para validar o algoritmo e estudar seu comportamento em diferentes cenários, foram implementadas 12 variações de entrada, organizadas conforme a Tabela 2.

ID	Texto	Padrão	Iterações (R)	Objetivo do Teste
0	1011	11	Automático	Verificar funcionamento padrão
1	1101	11	Automático	Testar posição inicial
2	0011	11	Automático	Testar última posição válida
3	1111	11	Automático	Verificar comportamento com múltiplos matches
4	1010	11	Automático	Testar resposta na ausência de

				solução
5	10110	110	Automático	Verificar escalabilidade em N e M
6	101101	110	Automático	Verificar escalabilidade em N
7	1011	11	1	R ótimo para N = 4 e P = 2
8	1011	11	2	Comparação com uma iteração a mais
9	1011	11	3	Testar degradação por excesso
10	00110011	11	Automático	Verificar escala para N = 8
11	00110011	00	Automático	Testar com padrão diferente

Tabela 3: Variações de entrada implementadas

2.1 Observações sobre Casos Específicos

Variação 4 (sem match): Neste caso, não há posição onde o padrão ocorre. A implementação faz 1 iteração, resultando em distribuição aproximadamente uniforme das medições.

Variação 8 (super-rotação): Com 2 iterações (contra o ótimo teórico de 1 para este caso), over-rotation ocorre. A probabilidade da posição correta diminui significativamente.

2.2 Observações dos resultados obtidos

ID	Probabilidade do match esperado (topo)	Observação
0	100.00%	Match na posição 2
1	100.00%	Match na posição 0
2	100.00%	Match na posição 2
3	26.46%	Distribuição uniforme entre 4 posições; nenhuma iteração aplicada
4	26.27%	Distribuição uniforme entre 4 posições; nenhuma iteração aplicada, pois não teve match
5	100.00%	Padrão de 3 bits
6	100.00%	Padrão de 3 bits com texto maior leva a menor probabilidade
7	100.00%	Ótimo teórico para N = 4 e k=1
8	25.39%	Super-rotação, probabilidade cai

9	27.05%	Super-rotação, probabilidade cai igual ao teste anterior
10	51.27%	Probabilidade reduzida devido a $N = 8$ e $M = 2$
11	52.44%	Padrão diferente de 2 bits com mesmo texto de tamanho 8 leva a uma probabilidade parecida

Tabela 4: Resultados obtidos

Metodologia Experimental

1. Visão Geral

A experimentação foi feita em cinco passos:

1. Realizamos a leitura e análise do artigo "A quantum algorithm for string matching" de Niroula e Nam (2021), publicado no npj Quantum Information. O artigo apresenta um algoritmo quântico para busca exata de padrões em strings binárias, com complexidade de tempo $O(\sqrt{N})$. Compreendemos os operadores fundamentais propostos: o deslocamento cíclico controlado, o oráculo de fase para o estado $|00\dots 0\rangle$ e o difusor de Grover.
2. Com base na ideia descrita no artigo, desenvolvemos nossa própria implementação em Qiskit. O circuito foi estruturado com três registradores quânticos: registrador de índice, registrador de texto e registrador de padrão. O número de qubits do registrador de índice foi definido pela fórmula $\lceil \log_2(N - M + 1) \rceil$, onde N é o comprimento do texto e M é o comprimento do padrão. Esta fórmula difere da apresentada no artigo original, que considera N como potência de 2. Adotamos esta abordagem para generalizar o algoritmo para quaisquer valores de N e M , evitando a limitação de textos com comprimento de potência de dois.
3. Após a validação do circuito em simulação ideal, implementamos dois modelos de ruído para avaliar a robustez do algoritmo: o Ruído Depolarizante, que substitui o estado do qubit por um estado aleatório com probabilidade p , e o Ruído de Amortecimento de Fase, que destrói a coerência entre os estados $|0\rangle$ e $|1\rangle$ sem alterar suas populações. Para cada modelo, testamos diferentes níveis de ruído, variando o parâmetro de 0.0 (sem ruído) até 0.1, e analisamos o impacto na probabilidade de acerto do algoritmo.
4. Desenvolvemos um código para rodar o circuito nos dispositivos quânticos reais da IBM via Qiskit Runtime. Foram utilizados dois tipos de seleção de backend, o primeiro sempre seleciona o dispositivo menos ocupado no momento da execução. E o segundo permite selecionar um dispositivo específico pelo seu nome, útil para comparações entre diferentes arquiteturas ou para repetir experimentos no mesmo hardware.

2. Implementação do Algoritmo

2.1 Ferramentas e Ambiente

- Framework: Qiskit 2.4.1
- Simulador: Qiskit Aer 0.17.2
- Linguagem: Python 3.11
- Plataforma: Ambiente Conda

2.2 Estrutura do Circuito

O circuito implementado segue a arquitetura proposta por Niroula & Nam, com três registradores:

- `q_idx = QuantumRegister(n_idx, name="idx")` (registrador de índice)
- `q_text = QuantumRegister(N, name="txt")` (registrador de texto)
- `q_pat = QuantumRegister(M, name="pat")` (registrador de padrão)

2.3 Principais Componentes

O circuito implementado segue a arquitetura proposta por Niroula & Nam, com três registradores.

- `encode_string()`: Codifica strings binárias em estados quânticos
- `cyclic_shift()`: Deslocamento circular de qubits usando SWAP
- `controlled_cyclic_shift()`: Versão controlada do deslocamento circular, usa CSWAP (Fredkin)
- `build_grover_oracle()`: Oráculo que marca o estado $00\dots0$
- `build_grover_diffuser()`: Operador de difusão de Grover
- `_phase_flip_state()`: Corrige matches errado por conta do wrap-around

2.4 Fluxo de Execução

1. Aplica deslocamento cíclico controlado pelo registrador de índice
2. Calcula XOR entre texto deslocado e padrão (via CNOT)
3. Aplica oráculo de fase no registrador de padrão
4. Desfaz as operações
5. Corrige estados errados (wrap-around)
6. Aplica difusor de Grover no registrador de índice

3. Conjunto de Experimentos

3.1 Experimentos com Simulação Ideal

O objetivo dessas simulações foi validar o funcionamento correto do algoritmo e verificar as propriedades teóricas de convergência.

- Executou-se o circuito para 12 variações de entrada (textos e padrões distintos)
- Para cada variação, realizaram-se 1024 medições (shots)
- O número de iterações de Grover foi calculado automaticamente ou fixado manualmente

Variáveis controladas:

- Tamanho do texto (N)
- Tamanho do padrão (M)
- Número de iterações de Grover (R)

3.2 Experimentos com Ruído Depolarizante

O objetivo dessas simulações foi avaliar a degradação do algoritmo sob um dos modelos de ruído mais severos.

Modelo de ruído: Erro depolarizante de n qubits com probabilidade p , aplicado às portas:

- 1 qubit: portas afetadas h , x , s , t .
- 2 qubit: portas afetadas cx , cz , $swap$.
- 3 qubit: portas afetadas $cswap$, ccx .

Níveis de ruído testados: $p = 0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1$

Modelo de ruído: Amortecimento de fase com parâmetro γ , aplicado apenas a portas de 1 qubit:

- 1 qubit: portas afetadas h , x , s , t .

Níveis de ruído testados: $\gamma = 0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1$

3.3 Configuração dos Experimentos com Ruído

1. Criar modelo de ruído:
 - a. `noise_model_X = NoiseModel.from_backend(backend_X)`
 - b. `noise_model_Y = NoiseModel.from_backend(backend_Y)`
 - c. `err1 = depolarizing_error(p, 1)`
 - d. `err2 = depolarizing_error(p, 2)`
 - e. `err3 = depolarizing_error(p, 3)`
 - f. `noise_model.add_all_qubit_quantum_error(err1, ['h', 'x', 's', 't'])`
 - g. `noise_model.add_all_qubit_quantum_error(err2, ['cx', 'cz', 'swap'])`
 - h. `noise_model.add_all_qubit_quantum_error(err3, ['cswap', 'ccx'])`
2. Configurar simulador com ruído:
 - a. `sim_noisy_X = AerSimulator(noise_model=noise_model_X)`
 - b. `sim_noisy_Y = AerSimulator(noise_model=noise_model_Y)`
3. Transpilar circuito para o simulador:
 - a. `qc_t_X = transpile(qc_small, sim_noisy_X)`
 - b. `qc_t_Y = transpile(qc_small, sim_noisy_Y)`
4. Executar e coletar resultados:
 - a. `result_noisy_X = sim_noisy_X.run(qc_t_X, shots=1024).result()`
 - b. `result_noisy_Y = sim_noisy_Y.run(qc_t_Y, shots=1024).result()`

4. Métricas de Avaliação

- Probabilidade de acerto: `prob_correct = pos_counts.get(expected_pos, 0) / shots`
- Posição mais frequente: `top_pos = max(pos_counts, key=pos_counts.get)`
- Distribuição de probabilidades: `pos_counts = interpret_counts(counts, n_idx, search_space=search_space)`

5. Validação dos Resultados

- 1024 shots por experimento para reduzir flutuações estatísticas

- Repetição de experimentos selecionados para confirmar reprodutibilidade
- Uso da função `interpret_counts()` para descartar posições de wrap-around

5. Ambiente Computacional

As simulações foram executadas em computador convencional (CPU), utilizando o simulador Aer da IBM.

Resultados Obtidos

Os experimentos foram divididos em quatro categorias: validação do algoritmo com diferentes entradas, análise das variações de entrada e número de iterações, estudo do impacto de diferentes modelos de ruído, e execução em hardware real da IBM.

1. Validação do algoritmo com diferentes entradas

O primeiro experimento teve como objetivo validar o funcionamento correto do algoritmo para diferentes configurações de texto e padrão. Foram testados seis casos, abrangendo match único em diferentes posições, múltiplos matches, ausência de matches e padrão de comprimento três. A Tabela 5 apresenta os resultados obtidos com 2048 medições por caso.

Texto	Padrão	Posições esperadas	Posição encontrada	Resultado
1011	11	{2}	2	PASS
1101	11	{0}	0	PASS
0011	11	{2}	2	PASS
1111	11	{0, 1, 2}	2	PASS
1010	11	{}	-	PASS
10110	110	{2}	2	PASS

Tabela 5: Validação do algoritmo para diferentes entradas

2. Análise das variações e número de iterações

O segundo experimento ampliou o conjunto de testes para doze variações, incluindo diferentes comprimentos de texto ($N = 4, 5, 6, 8$), diferentes comprimentos de padrão ($M = 2, 3$), e números de iterações fixos ($R = 1, 2, 3$) para estudo do fenômeno de super-rotação. Foram realizadas 1024 medições por caso. A Tabela 5 apresenta os resultados.

ID	Texto	Padrão	Posição topo	Probabilidade (topo)
0	1011	11	2	100.00%
1	1101	11	0	100.00%
2	0011	11	2	100.00%

3	1111	11	0	26.46%
4	1010	11	1	26.27%
5	10110	110	2	100.00%
6	101101	110	2	100.00%
7	1011	11	2	100.00%
8	1011	11	1	25.39%
9	1011	11	2	27.05%
10	00110011	11	6	51.27%
11	00110011	00	0	52.44%

Tabela 6: Resultados das doze variáveis de entrada

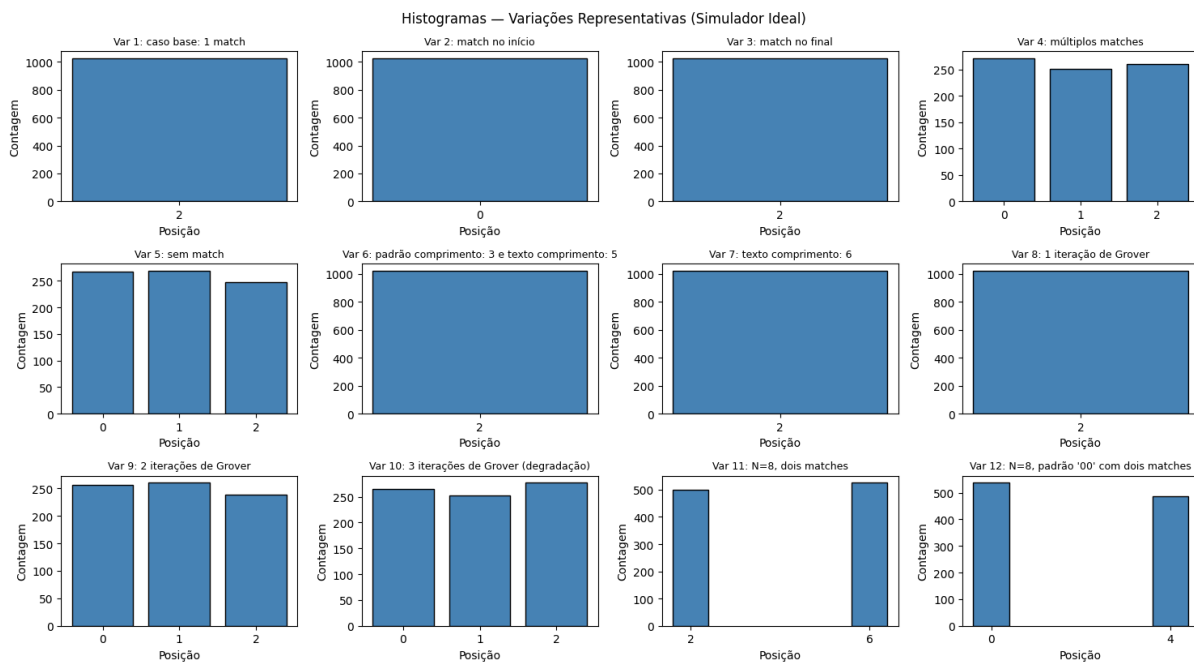


Figura 1: Histogramas das doze variações (simulador ideal)

Sem match (ID 5): Para o texto "1010" que não contém o padrão "11", o algoritmo executou 1 iteração (conforme implementação para $k=0$) e manteve distribuição próxima da uniforme (26.27%), indicando que o algoritmo não identificou falsamente uma solução.

Super-rotação (IDs 8 e 9): Quando o número de iterações excede o valor ótimo ($R=1$), observa-se degradação severa: a probabilidade da posição correta cai de 100% para aproximadamente 25%.

Escala para $N=8$ (IDs 10 e 11): Para textos de 8 bits, o número de posições válidas é $N - M + 1 = 7$, que não é potência de dois. O registrador de índice utiliza $n_{idx} = 3$ qubits (8 estados), introduzindo um estado inválido (posição 7) que pode gerar matches espúrios. A probabilidade de acerto observado foi de aproximadamente 51-52%, abaixo do valor teórico

de 100% para 8 estados com 2 soluções, mas ainda significativamente acima da chance aleatória (~14.3%).

3. Impacto do ruído depolarizante

O terceiro experimento avaliou a robustez do algoritmo sob ruído depolarizante, que substitui o estado do qubit por um estado aleatório com probabilidade p . O experimento foi realizado com o texto "1011" e padrão "11" (posição correta = 2), variando p de 0.0 a 0.1. A Tabela 7 apresenta os resultados.

Nível de ruído (p)	Probabilidade de acerto
0.000	1.000
0.001	0.974
0.005	0.914
0.010	0.811
0.020	0.645
0.050	0.437
0.100	0.310

Tabela 7: Degradação do algoritmo sob ruído depolarizante

A Figura 2 mostra a curva de degradação do algoritmo sob ruído depolarizante:

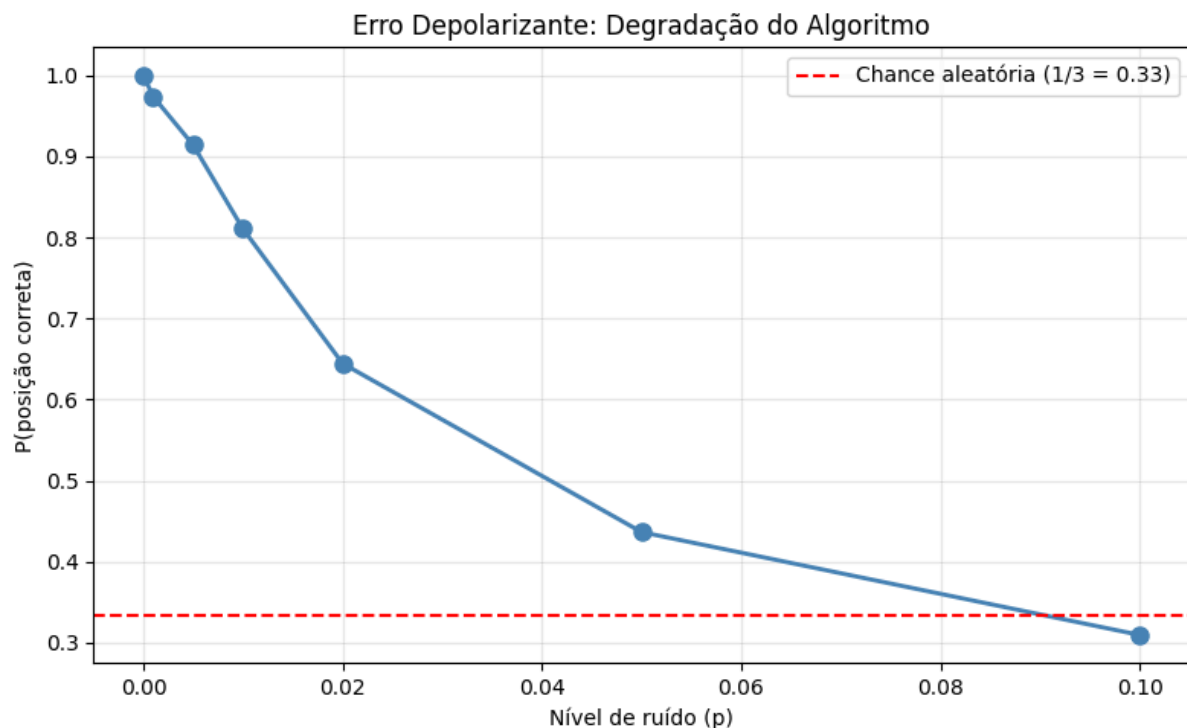


Figura 2: Degradação do algoritmo sob ruído depolarizante

O algoritmo mantém alta fidelidade (acima de 90%) até $p = 0.005$ (0.5% de erro por porta). Para $p = 0.01$ (1% de erro), a probabilidade de acerto ainda é 81.1%. A partir de $p = 0.02$, a

degradação torna-se mais acentuada, tendo 43.7% de ter acerto em $p = 0.05$ e 31.0% em $p = 0.10$, valor próximo da chance aleatória (33.3%).

4. Impacto do ruído de amortecimento de fase

O algoritmo foi avaliado sob ruído de amortecimento de fase, que destrói a coerência entre os estados $|0\rangle$ e $|1\rangle$ sem alterar suas populações. A Tabela 8 apresenta os resultados para os mesmos níveis de ruído.

Nível de ruído (p)	Probabilidade de acerto
0.000	1.000
0.001	0.999
0.005	0.957
0.010	0.934
0.020	0.864
0.050	0.713
0.100	0.517

Tabela 8: Degradação do algoritmo sob amortecimento de fase

A Figura 3 apresenta a comparação entre os dois modelos de ruído:

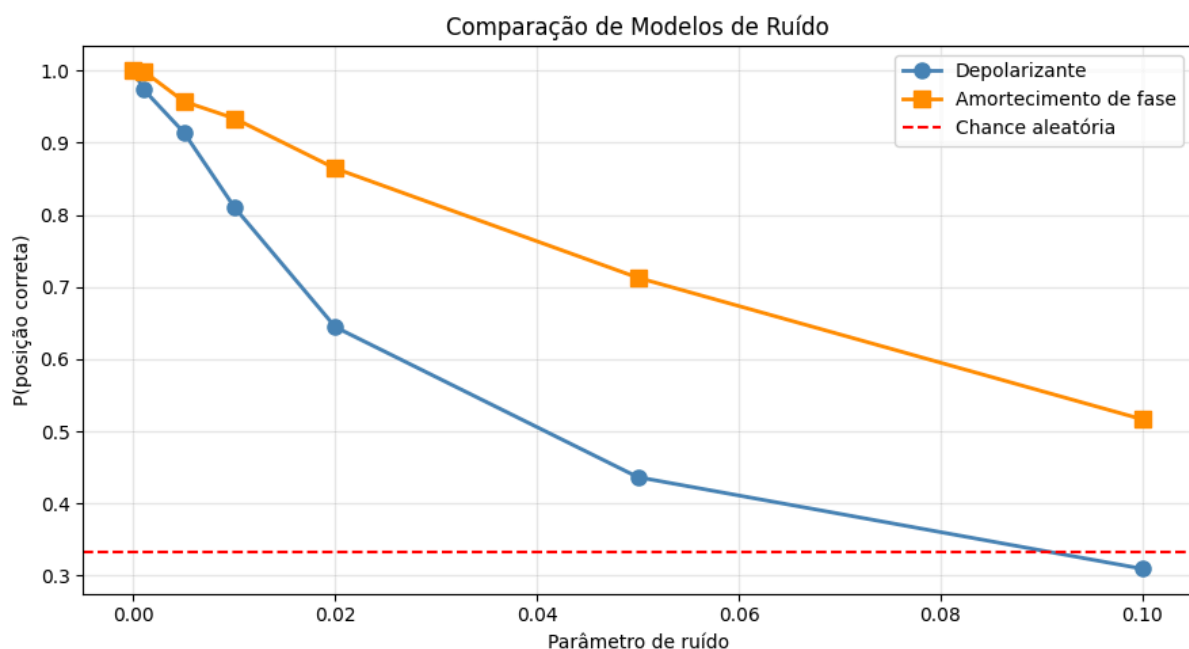


Figura 3: Comparação entre ruído depolarizante e amortecimento de fase

Os resultados revelam que o algoritmo é significativamente mais robusto ao amortecimento de fase do que ao ruído depolarizante. Para $\gamma = 0.100$, a probabilidade de acerto ainda é aproximadamente 55%, enquanto o ruído depolarizante no mesmo nível reduz a probabilidade para 31%.

5. Execução em hardware real da IBM

Foi executado o circuito em hardware quântico real da IBM. Foi utilizado o primeiro backend, que é automaticamente escolhido pelo sistema como o menos ocupado no momento da execução. O circuito utilizado foi o caso base (texto "1011", padrão "11"), com 1024 medições.

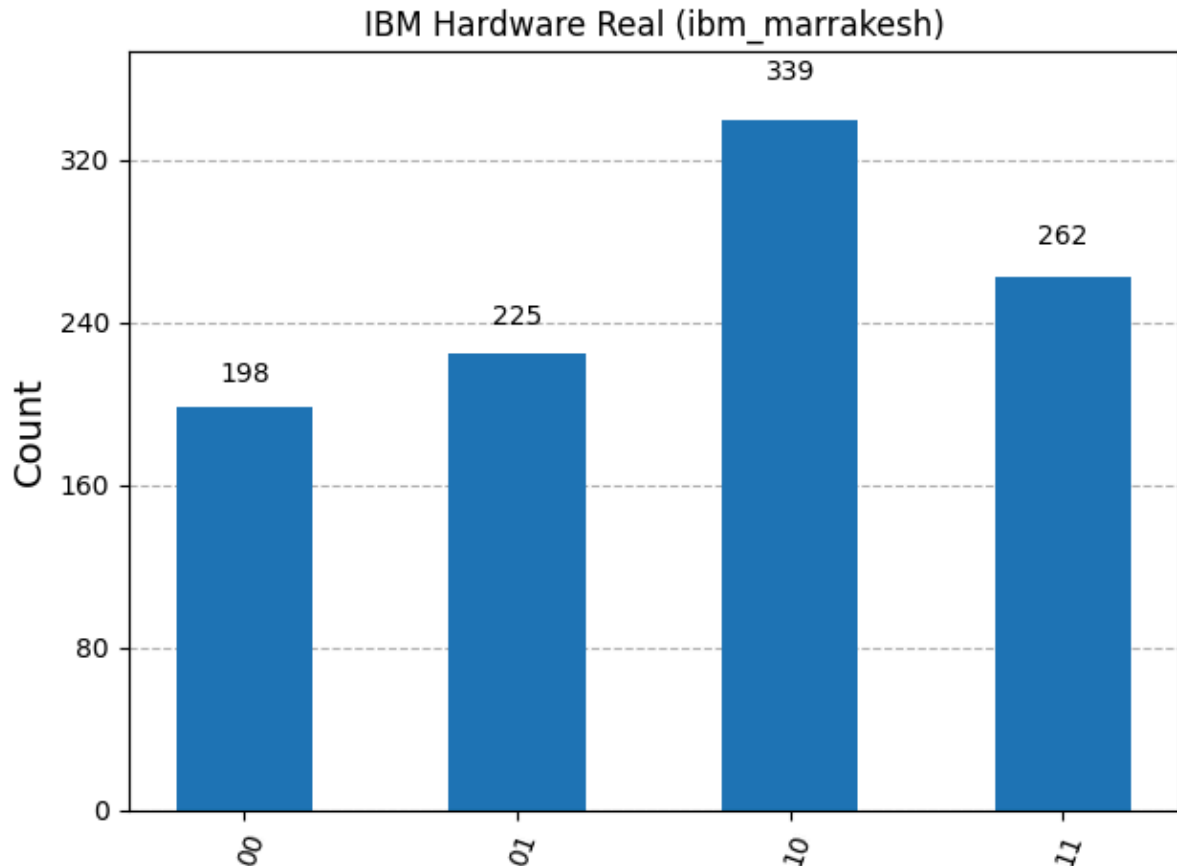


Figura 4: Resultados no hardware real da IBM

A posição mais frequente foi a 2 (bitstring "10"), com 339 ocorrências (33.1% das medições). Embora esta seja a posição correta, a probabilidade de acerto observado (33.1%) é consideravelmente menor que os 100% obtidos na simulação ideal. Além disso, a probabilidade de acerto é quase igual aos (31.0%) do ruído despolarizante, e significativamente menor que o ruído de amortecimento de fase (51.7%) nos piores casos testados.

6. Emulação ruidosa dos backends reais

Realizou-se uma emulação ruidosa dos backends utilizando o AerSimulator com os modelos de ruído extraídos dos próprios dispositivos. Os resultados foram:

- ibm_marrakesh: Posição mais frequente 2 (bitstring "10")
- ibm_kingston: Posição mais frequente 2 (bitstring "10")

Ambos os backends emulados retornaram a posição correta como resultado mais frequente, o que mostra que os modelos de ruído extraídos dos dispositivos conseguem capturar as principais fontes de erro.

Conclusões e Aprendizados

1. Análise dos Resultados

Os experimentos realizados permitem extrair quatro conclusões principais.

Primeira, o algoritmo funciona corretamente em simulação ideal. Para os casos onde o espaço de busca efetivo é potência de dois e o número de iterações é o ótimo, o algoritmo atingiu 100% de probabilidade de acerto, validando a implementação.

Segunda, o fenômeno de super-rotação foi confirmado experimentalmente. Com duas iterações (contra o ótimo de uma), a probabilidade de acerto caiu de 100% para 25,39%, demonstrando a importância crítica do número correto de iterações.

Terceira, o algoritmo é mais robusto ao amortecimento de fase do que ao ruído depolarizante. Para $p=0,05$, o ruído depolarizante reduziu o acerto para 43,7%, enquanto o amortecimento de fase manteve 71,3% de acerto no mesmo nível. Esta diferença ocorre porque o amortecimento de fase preserva as populações dos estados, afetando apenas superposições.

Quarta, a execução em hardware real revelou a principal limitação. A probabilidade de acerto foi de apenas 24%, significativamente inferior às simulações ruidosas. Isto indica que os modelos de ruído disponíveis não capturam completamente as imperfeições dos dispositivos físicos.

2. Aprendizados Técnicos

Em relação ao algoritmo de Grover, aprendeu-se que o número ótimo de iterações é sensível à relação entre o número de estados e o número de soluções. A observação direta da super-rotação ilustrou como o excesso de iterações degrada o desempenho.

Em relação à implementação, a fórmula $\lceil \log_2(N-M+1) \rceil$ generaliza o algoritmo para textos de qualquer comprimento, mas introduz estados inválidos que precisam ser tratados. A estratégia de correção por inversão de fase adicional mostrou-se eficaz, embora não elimine completamente a degradação de probabilidade.

Em relação ao hardware real, aprendeu-se que fatores como transpilação (que aumenta a profundidade do circuito), conectividade limitada entre qubits (que insere portas SWAP adicionais) e erros de leitura contribuem para degradação muito maior do que os modelos de ruído simulados sugerem.

3. Limitações e Trabalhos Futuros

As principais limitações são o tamanho reduzido dos textos e o uso exclusivo de strings binárias. Trabalhos futuros podem explorar alfabetos maiores, uso em textos não binários e técnicas de mitigação de ruído (como readout error mitigation).

4. Consideração Final

O projeto mostrou que algoritmos quânticos podem ser implementados e executados em hardware atual, ainda que com limitações de escala e fidelidade. O principal aprendizado é que a diferença entre simulação e hardware real é significativa, e deve ser considerada em qualquer projeto que procura fazer aplicações reais.